Software Timers:

Sometimes, we don't need a heavy, dedicated task that loops forever using vTaskDelay. Instead, we just want a specific piece of code to execute at a precise time interval (like checking a battery level every 30 seconds) or execute exactly once after a set delay (like a timeout fallback).

FreeRTOS handles this via a dedicated, low-overhead module called Software Timers.

Two Types of Software Timers

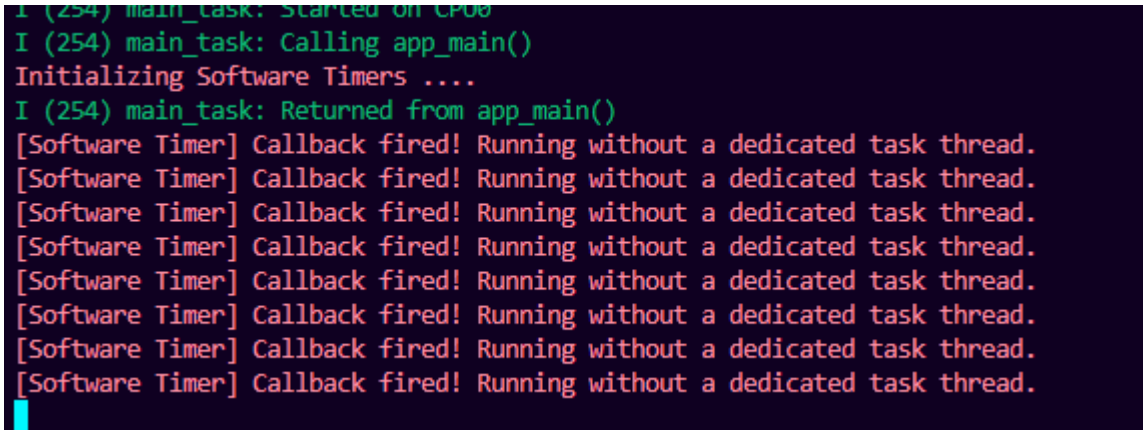
1. One-Shot Timers: These start, wait for their specified period, execute their callback function exactly once, and then stop automatically.
2. Auto-Reload Timers: These wait for their period, execute their callback, and immediately reset themselves to run again forever in a recurring loop.

Code:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/timers.h"

TimerHandle_t my_periodic_timer;

void timer_callback(TimerHandle_t xTimer){
    printf("[Software Timer] Callback fired! Running without a dedicated task
thread. \n");
}
// Creating the software timer
void app_main(void){
    printf("Initializing Software Timers ....\n");

    my_periodic_timer = xTimerCreate(
        "Periodic Timer",
        (1500 / portTICK_PERIOD_MS),
        pdTRUE,
        (void *) 0,
        timer_callback
    );
// Starting the timer if allocation was successful
if(my_periodic_timer != NULL){
    xTimerStart(my_periodic_timer, 0);
} else{
    printf("Failed to create software timer! \n");
}
}
```

```
my_periodic_timer = xTimerCreate( "Periodic Timer", // Debug text name
(1500 / portTICK_PERIOD_MS),                        // Timer period (1.5 seconds)
pdTRUE,                 // Auto-Reload flag (pdTRUE = Auto-Reload, pdFALSE = One-Shot)
(void *) 0,             // Timer ID unique tracking parameter
timer_callback          // The callback function pointer to execute );
```

Output:



 Notice that app_main launches the timer and then exits completely.

 There are no active xTaskCreate statements in this file! FreeRTOS manages all software timers inside a single hidden, internal system task called the Timer Daemon Task. This keeps the system's RAM footprint small because you don't have to allocate individual stacks for simple periodic events.